

Design of a 32-bit, dual pipeline superscalar RISC-V processor on FPGA

Gokulan T, Akshay Muraleedharan, Kuruvilla Varghese, Senior Member, IEEE
Electronic Systems Engineering, Indian Institute of Science, Bangalore, INDIA,
Email: {gokulant, makshay, kuru}@iisc.ac.in

Abstract—A 40 MHz, 32-bit, 5-stage dual-pipeline superscalar processor based on RISC-V Instruction Set Architecture is presented. It supports integer, multiply-divide and atomic read-modify-write operations. The proposed system implements in-order issuing of instructions. The design incorporates a dynamic branch prediction unit, memory subsystem with virtual memory, separate instruction cache and data cache, integer and floating point execution units, interrupt controller, error control module, and a UART peripheral. The interrupt controller supports four levels of preemptive priority, which is programmable for individual interrupts. Error control module provides single error correction and double error detection for the main memory. Wishbone B.3 bus standard is adopted for on-chip communication. The processor is implemented on Virtex-7 XC7VX485TFFG1761-2 FPGA based board. CoreMark and Dhrystone benchmark values for the design are 3.84/MHz and 1.0603 DMIPS/MHz respectively.

I. INTRODUCTION

Processors are mainly evaluated by their speed, performance and Instruction Set Architecture (ISA). An open source ISA allows more freedom for innovation with lower cost and more flexibility unlike most of the commercial ISAs. In other words, a freely open ISA can be easily worked upon for a specific application and workload in an efficient manner. RISC-V is the chosen ISA for the proposed design because of its enhanced features as compared to OpenRISC, SPARC and others [2].

The RISC-V ISA is defined as a base integer ISA (RV32I), which must be present in any implementation, plus optional extensions to the base ISA. Standard extension is RV32G, which includes Integer (I), Multiply/Divide (M), Atomic (A), Single Precision Floating Point (F) and Double Precision Floating Point (D) instructions [4].

UC Berkeley has already developed silicon implementations of RISC-V, and there are external projects underway in many countries as well. One such variant of the RISC-V ISA is “Rocket core” [5], which is about the same performance level as an ARM A5 when implemented in the same process technology, but is 64-bit instead of 32-bit, and has a better form factor and dynamic power consumption compared to 32-bit ARM core. In the past couple of years, more implementations of RISC-V ISA has come out. ZScale/VScale is a RISC-V ISA based processor with RV32IM architecture [6]. FPGA Prototype of a 32-bit single cycle RISC-V processor is presented in [7]. The paper [8] presents a RISC-V compatible processor IP, which involves 32-bit RV32IMA architecture with a 5-stage pipeline. Many

companies like NVIDIA, Western Digital, etc. also have their ongoing research on RISC-V open platforms [9] [10].

In this paper, a 32-bit superscalar processor with two 5-stage pipelines is presented. It implements the RV32IMAFD extension of RISC-V ISA. This work is focussed on improving the throughput by incorporating additional functionalities to the scalar processor described in [8]. An additional pipeline is implemented and the complexities that come with this is managed. A dynamic branch prediction algorithm is implemented. This RISC-V processor is suitable for applications in miniature embedded processors, IoT applications, machine learning, etc. It can be used in military applications, especially defense, because it is an open architecture and can be verified for malicious code that can harm its operation.

The rest of the paper is organized as follows. Section II describes the processor architecture in brief. Section III discusses the design of all blocks. Section IV describes various cases for verification. Results of FPGA implementation are given in Section V, followed by conclusion in Section VI.

II. PROPOSED PROCESSOR ARCHITECTURE

The processor is a 32-bit, 5-stage, two-way in-order superscalar pipeline architecture with support for RV32IMAFD instructions. Fig. 1 shows the dual-pipeline superscalar architecture for the processor. The pipeline stages are Fetch (IF), Decode (ID), Execute (EX), Memory (MEM), and Write-back (WB).

In the IF stage, program memory address in program counter (PC) is generated. This PC value is used to fetch two consecutive instructions from Instruction Cache (I-Cache) and then send it to the ID stage. Based on the dependencies between instructions, Instruction Issuing Unit (IIU) issues either one or two instructions to the pipeline. The architecture supports parallel issuing of two integer instructions or one integer and one floating point instructions. The decode stage deciphers the instruction and generates select signals for the multiplexers for data forwarding in the EX stage. In case of Atomic Memory Operations (AMO), the ID stage is responsible for reading data from the data memory.

Operands from the ID stage are given to the EX stage. The EX stage has two integer and one floating point units to execute at most two instructions (two integer or one integer and one floating point instruction) in the same clock cycle. Forwarding lines toggle the multiplexers so that either the

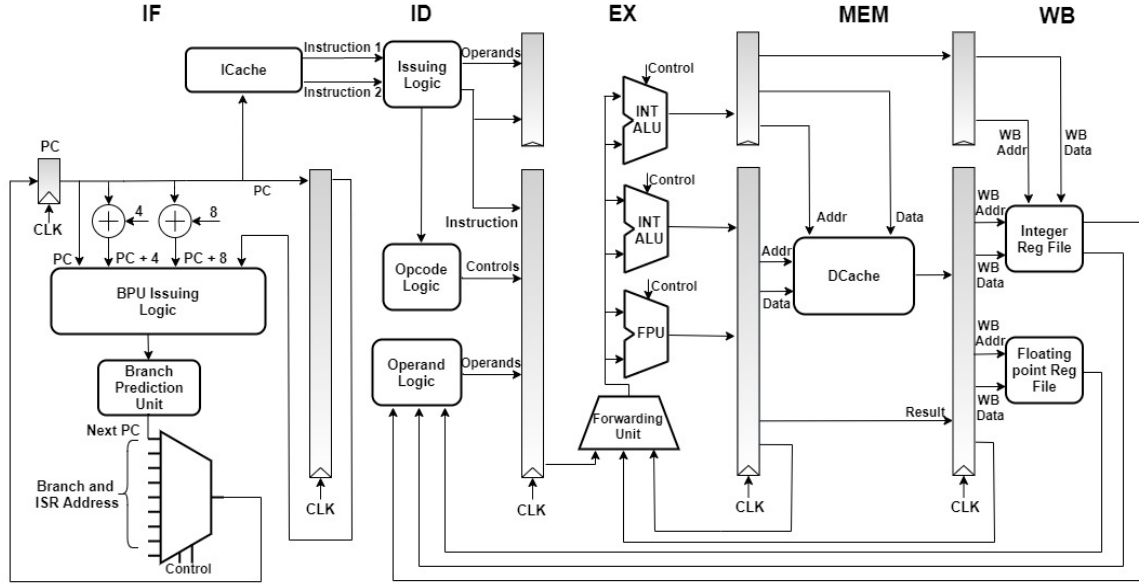


Fig. 1: 5-Stage Two-way In-order Superscalar Pipeline Architecture

forwarded data or the data from register/memory is used in EX stage. IF and ID stages are stalled when multiply/divide or floating point instructions enter the execution stage because its execution takes variable number of clock cycles. Results from the EX stage are given to the memory stage to perform data transactions for load/store/atomic instructions. In other cases, results will be forwarded to the WB stage. In the WB stage, register file is updated, if needed. Separate register files are implemented for integer and floating point instructions. The processor has 32, 32-bit registers for integer operations and 32, 64-bit registers for floating point operations.

A few conventions have been followed in the design to reduce complexities and resource utilization. Two instructions are issued to the pipeline, only if the second instruction does not have any dependency on the first instruction. In case of dependencies, only the first instruction is issued and the second instruction is held back at the ID stage. Two memory access instructions (load/store or atomic) or two branch instructions are never issued together. Load, branch, multiply/divide and control status register (CSR) access instructions are issued only through the first pipeline.

Wishbone B.3 bus standard [12] by OpenCores is implemented in the system. A branch prediction unit for the superscalar issuing (considering dependencies) is implemented based on dynamic branch prediction algorithm [13].

III. DESIGN IMPLEMENTATION

A. Instruction Memory Subsystem

Instruction memory subsystem controls flow of instructions, with the help of IF stage, from main memory to ID stage. It consists of I-Cache and Instruction Translation Lookaside Buffer (ITLB).

I-Cache is 8 KB, 2-way set associative with block size 8-words (32-bytes). It is accessed through the IF stage of the

processor pipeline. I-Cache has two read ports, each having a port size of 1-word in order to support for dual issuing of instructions. Fig. 2 shows the block diagram of interface to the I-Cache.

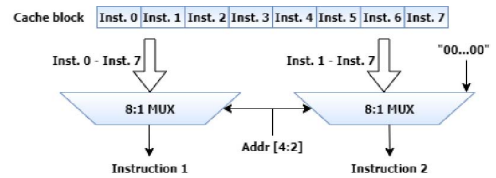


Fig. 2: Instruction Cache Interface

ITLB contains Content Addressable Memory (CAM), replacement block and a control unit to manage TLB operations. If the request to CAM is a hit, then page table entry is given as output. In case of miss, control unit will start bus-transaction with main memory and if CAM is not empty, then an existing entry is replaced based on Economic Value Added (EVA) [14] policy.

B. Data Memory Subsystem

Data memory subsystem controls the flow of data, with the help of ID and MEM stages, from the main memory to EX and WB stages. It consists of Data Cache (D-Cache) and Data Translation Lookaside Buffer (DTLB). D-Cache is 8 KB, 2-way set associative with block size of 8 words. DTLB has 32 entries.

D-Cache has two memory access ports. One port is used for regular load/store operation and atomic (read-modify-write) write operation. The other port is used for atomic (read-modify-write) read operation. DTLB is exactly similar to ITLB.

C. Instruction Issuing Unit

Instruction issuing unit (IIU) is implemented in the ID stage. IIU decodes both the instructions coming from the I-Cache and compares the value of operands in each instruction to find dependency between the instructions. Whenever dependency arises, the second instruction will be stored in a hold register and a “rollback” signal will be asserted to adjust next-PC value. The held instruction will be issued in the next clock cycle. Fig. 3 shows the block diagram of IIU.

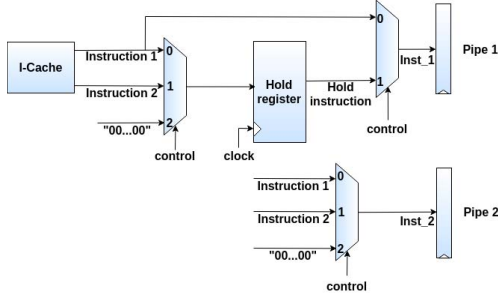


Fig. 3: Instruction Issuing Unit

D. Next-PC Logic

Whenever the “rollback” signal is asserted in the ID stage, next PC value will be the address of next instruction ($PC + 4$), otherwise it is the address of the instruction after the next instruction ($PC + 8$). Behaviour of superscalar instruction fetching mechanism is illustrated in Table I assuming only the first two instructions are having dependency.

Cycle	IF Stage	ID Stage
1	Current Instructions : I0, I1 Next Instructions : I2, I3 rollback = 0	First Instruction : Null Second Instruction : Null Hold Instruction : Null
2	Current Instructions : I2, I3 Next Instructions : I3, I4 rollback = 1	First Instruction : I0 Second Instruction : Null Hold Instruction : I1
3	Current Instructions : I3, I4 Next Instructions : I5, I6 rollback = 0	First Instruction : I1 Second Instruction : I2 Hold Instruction : Null

TABLE I: Instruction Fetching Mechanism assuming only the first two instructions (I0 and I1) have dependencies.

Next PC calculation will be different in case of branch instructions. In the IF stage, if the BPU has predicted the target PC value of a branch instruction, the predicted PC from the BPU will be loaded. If the prediction goes wrong, then the correct value will be loaded from the MEM stage. Fig. 4 shows the block diagram of the next-PC logic implemented in the IF stage.

E. Superscalar Branch Prediction

Superscalar Branch Prediction Unit (BPU) is implemented in the IF stage. The BPU predicts the result of branch instructions based on the 2-bit dynamic branch prediction algorithm in the IF stage, before waiting for the branch instruction to complete. It takes one clock cycle to give the

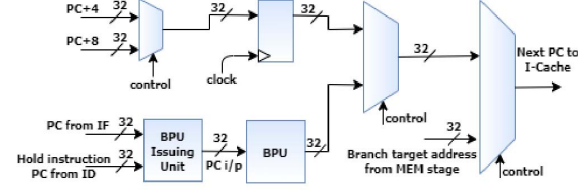


Fig. 4: Next-PC Logic

prediction result. Branch prediction check is done in EX stage of the pipeline. If prediction goes wrong, then correct PC value is loaded in the next clock cycle and IF, ID and EX stages are flushed.

Branch prediction issuing unit is implemented in the IF stage for feeding the PC value to the BPU. The PC value from the hold register in the ID stage is fed to the BPU, if the held instruction is a branch. Otherwise the PC value from the IF stage is fed to the BPU. This is shown in Fig. 5.

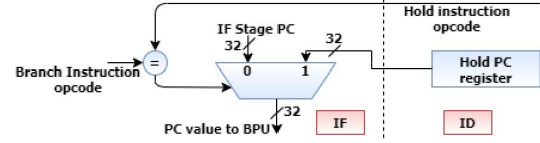


Fig. 5: Branch Prediction Issuing Unit

F. Forwarding Unit

Forwarding unit is implemented in the EX stage to handle the data hazards arising from the dependencies between instructions. Whenever a hazard condition arises both from the MEM and WB stage instructions, data from the MEM stage is forwarded since MEM stage is having the latest instruction. If instructions in both the pipelines make a hazard situation, then the data from the second pipeline takes priority, since at any stage, the instruction in the second pipeline is the latest one. Since the superscalar pipeline architecture consists of two integer and one floating point execution units, three forwarding units are implemented in the EX stage. One of those forwarding unit is shown in Fig. 6. The output from forwarding unit is fed to the ALU.

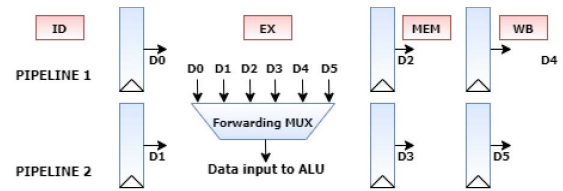


Fig. 6: Forwarding unit

G. Interrupt Handler

The interrupt controller comprises of interrupt edge detector, control logic, interrupt interface, and register bank. Interrupt edge detector receives interrupts from external/internal sources

and generates interrupt pulse. Control logic takes the inputs, sorts them according to priority and then updates the Register bank. It sends a request to the interrupt interface, which initiates interrupt processing. Interrupt interface, then saves the current status of the pipeline and acknowledges control logic through an acknowledge signal. At the end of these instructions, fetch stage is released from stall and PC is set to interrupt service routine (ISR) address. On completion of ISR, interrupt interface takes control of the pipeline to restore previous status.

The design supports partial hardware context saving. The PC value is saved by the hardware itself, but the registers should be saved by the software.

IV. VERIFICATION

Verification is done for checking whether all types of instructions like integer, floating point, multiply/divide and load/store are executed properly and data hazards are handled by the forwarding unit. Some standard C codes are used for testing the functionality of superscalar processor. Standard C codes are compiled using RISC-V toolchain [15] and running special python scripts and corresponding .coe file is generated. This .coe file contains the binary code of the C program, which is made part of the FPGA programming bitstream by the FPGA design tool. While configuring FPGA, instruction memory is initialized with this data, and processor executes this program after configuration of the FPGA.

V. RESULTS

The processor is designed in Verilog using Xilinx Vivado 2018.2 and is implemented on Virtex-7 XC7VX485TFFG1761-2 FPGA based board. The maximum achievable frequency with this FPGA is 40 MHz. Resource utilization in Virtex-7 FPGA after implementation is shown in Table II.

Module	LUTs	Registers	Block RAM
Pipeline	30125	8532	8.5
D-Cache	10631	10039	20
I-Cache	7739	6409	9
Integer Register File	2862	1088	0
Main Memory	144	4	64
UART	496	373	0

TABLE II: Resources utilization in Virtex-7 FPGA for Superscalar design

CoreMark and Dhrystone benchmark standard programs have been run on the processor to evaluate the performance. The values are 3.84/MHz and 1.0603 DMIPS/MHz respectively. The CoreMark value is compared with that of some standard ARM Cortex-M series processors available in market. Performance of our processor is better than the ones compared against. It is shown in Table III.

VI. CONCLUSION

A 32-bit, single core, dual-pipeline, superscalar architecture is discussed in this paper. The processor is implemented on Virtex-7 FPGA and results are shown in terms of

Processor	CoreMark/MHz
RISC-V (This work)	3.84
Cortex-M4 (NXP Kinetis K70)	3.40
Cortex-M3 (STM32L152)	3.34
Cortex-M0+ (Atmel SAM D20)	2.46
Cortex-M0 (STM32F051C8)	2.33

TABLE III: CoreMark result comparison

resources, frequency and CoreMark benchmark value. It is extensively tested and verified for its branch prediction, interrupt handling and for all corner cases. Further, dual-issue pipeline architecture can be modified to multiple-issue pipeline with more number of execution units. Out of order execution can be implemented for higher utilization of functional units, less stalling and better performance. Linux can be ported on the system with minimal effort by implementing supervisor privilege level, as virtual memory is already part of the system. Multi-core system can be designed using this single core and can be made as a customizable IP with variable configuration.

REFERENCES

- [1] Andrew Waterman, "Design of the RISC-V Instruction Set Architecture", PhD thesis, EECS Department, University of California, Berkeley, Jan 2016.
- [2] Krste Asanovic and David A. Patterson, "Instruction sets should be free: The case for RISC-V", Technical Report UCB/EECS-2014-146, EECS Department, University of California, Berkeley, Aug 2014.
- [3] <https://riscv.org/>
- [4] Andrew Waterman, Yunsup Lee, David A. Patterson, and Krste Asanovic, "The RISC-V Instruction Set Manual, Volume I: User-level ISA, version 2.0", Technical Report UCB/EECS-2014-54, EECS Department, University of California, Berkeley, May 2014.
- [5] Albert Magyar, Yunsup Lee, and Albert Ou, "Z-scale: Tiny 32-bit risc-v systems with updates to the rocket chip generator", The International House, Berkeley, 2015.
- [6] Verilog version of z-scale, vscale, <https://github.com/ucbar/vscale.2016>.
- [7] Don Kurian Dennis, Ayushi Priyam, Sukhpreet Singh Virk, Sajal Agrawal, Tanuj Sharma, Arijit Mondal, and Kailash Chandra Ray, "Single Cycle RISC-V Micro Architecture Processor and its FPGA Prototype", 7th International Symposium on Embedded Computing and System Design (ISED), 2017.
- [8] Suseela Budi, Pradeep Gupta, Kuruvilla Varghese, and Amrutur Bharadwaj, "A RISC-V ISA Compatible Processor IP for SoC", International Symposium on Devices, Circuits and Systems (ISDCS) 2018.
- [9] Xie, Joe (July 2016), NVIDIA RISC V Evaluation Story, 4th RISC-V Workshop, Youtube.
- [10] Western Digital To Accelerate The Future Of Next-Generation Computing Architectures For Big Data And Fast Data Environments. Western Digital, 2017-11-28.
- [11] S. M. Bhagat and S. U. Bhandari, "Design and Analysis of 16-bit RISC Processor", Fourth International Conference on Computing Communication Control and Automation (ICCUBEA), 2018.
- [12] OPENCORES.ORG. WISHBONE System-On-Chip (SoC) Interconnection Architecture for Portable IP Cores, Revision B.3., 2015.
- [13] H. K. Kim, H. S. Kim, C. M. Eun, H. H. Cho, and O. H. Jeong, "A high-performance branch predictor design considering memory capacity limitations", International Conference on Circuits, System and Simulation (ICSSS), 2017.
- [14] N. Beckmann and D. Sanchez, "Maximizing Cache Performance Under Uncertainty", IEEE International Symposium on High Performance Computer Architecture (HPCA), 2017.
- [15] RISC-V Tool Chain. <https://github.com/riscv/riscv-gnu-toolchain>.
- [16] John Paul Shen and Mikko H. Lipasti, "Modern processor design fundamentals of superscalar processors", Tata McGraw-Hill, 2001.
- [17] CoreMark. <https://www.eembc.org/coremark/index.php>